PANDAS: Peer-to-peer, Adaptive Networking Allowing Data Availability Sampling within Ethereum Consensus Timebounds

Matthieu Pigaglio
UCLouvain
Belgium

Onur Ascigil
Lancaster University
United Kingdom

Michał Król
City, University of London
United Kingdom

Sergi Rene
Datahop Labs
United Kingdom

Felix Lange
Ethereum Foundation
Germany

Kaleem Peeroo
City, University of London
United Kingdom

Ramin Sadre
UCLouvain
Belgium

Vladimir Stankovic
City, University of London
United Kingdom

Etienne Rivière
UCLouvain
Belgium

Abstract

Layer-2 protocols such as rollups can help address Ethereum's throughput limits. An efficient data availability layer is key for layer-2 support in Ethereum, but broadcast methods do not scale. A promising approach is the selective distribution of layer-2 data and its verification by data availability sampling (DAS). Integrating DAS with Ethereum consensus is, however, a challenge, as data must be shared and sampled within 4 seconds of each consensus slot.

We propose PANDAS, a practical approach to integrating DAS with Ethereum without modifying Ethereum's core protocols. PANDAS disseminates layer-2 data and samples its availability using lightweight, direct exchanges. Its design accounts for message loss, node failures, and unresponsive participants. Our evaluation in a 1,000-node cluster and simulations for up to 20,000 peers show that PANDAS allows layer-2 data dissemination and sampling under planetary-scale latencies within the 4-second deadline.

CCS Concepts

• **Computer systems organization** → **Peer-to-peer architectures**; **Fault-tolerant network topologies**.

Keywords

Ethereum, Data Availability Sampling, Peer-to-peer, Performance

ACM Reference Format:
Matthieu Pigaglio, Onur Ascigil, Michał Król, Sergi Rene, Felix Lange, Kaleem Peeroo, Ramin Sadre, Vladimir Stankovic, and Etienne Rivière. 2025. PANDAS: Peer-to-peer, Adaptive Networking Allowing Data Availability Sampling within Ethereum Consensus Timebounds. In *26th ACM Middleware Conference (Middleware '25), December 15–19, 2025, Nashville, TN, USA.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3721462.3770769



This work is licensed under a Creative Commons Attribution 4.0 International License.
Middleware '25, December 15–19, 2025, Nashville, TN, USA
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1554-9/25/12
https://doi.org/10.1145/3721462.3770769

1 Introduction

Ethereum, the largest blockchain supporting smart contracts, currently supports adding less than a few tens of transactions per second to its main (layer-1) chain. Complementarily to layer-1 scalability improvements [32, 54], expanding support for layer-2 protocols [30] is now a priority for the Ethereum community [22].

Layer-2 protocols such as *side chains* and *rollups* have the potential to process a large number of transactions [30, 37]. These protocols periodically produce compressed or batched layer-2 transaction data, which they make available via layer-1 data availability mechanisms. For instance, participants in an optimistic rollup can download this data, verify its correctness, and submit fraud proofs [25, 39, 49].

The throughput of layer-2 protocols depends on how much data they can attach to layer-1 blocks. Previously, the only solution was adding layer-2 data as costly *calldata* transactions. These transactions competed for permanent block space with other layer-1 transactions, such as ETH transfers. In March 2024, the EIP-4844 proposal [10] introduced the notion of *blobspace*. Layer-2 data can now be shared as opaque *binary objects* (blobs). Blobs are broadcast separately and referenced by *blob-carrying* transactions in the block, which include cryptographic commitments to referenced blob content. Nodes participating in consensus verify these commitments and make blob data available to layer-2 participants for a limited time (4,096 epochs, ~18 days). While improving over *calldata* transactions in terms of costs and supported volume, EIP-4844 still requires blob data to be broadcast and received by all nodes.

A key target of Ethereum is to scale data availability support. The *Danksharding* roadmap [23] aims to support up to 32 MB blob data attached to every layer-1 block. To avoid broadcasting this volume of data globally, blob data is erasure-coded, split, and distributed as collections of *cells*, so that each node holds only a fraction of the data. This shift introduces a new challenge: no single node can independently verify the availability of the full blob. This verification remains, nonetheless, necessary to attest to the validity of the corresponding block. To address this, Ethereum plans to adopt *data availability sampling* (DAS), wherein nodes collect random *sample* cells from the network until they reach overwhelming confidence

that the complete data can be reconstructed. The Danksharding target parameters imply sending 140 MB of erasure-coded cells to the network, with each node randomly sampling 73 cells (40 KB).

Integrating DAS and Ethereum consensus is a challenge. The need for each node to collect randomly selected samples results in a multitude of exchanges over high-latency links. At the same time, Ethereum's consensus imposes tight time constraints. A new block is generated every 12 seconds. A committee must validate each new block within the first four seconds after it is created. Validating the availability of associated blob data *after* this deadline (named the *trailing* fork-choice rule [16]) may lead to reverting validation and consensus decisions due to its unavailability. Modifying Ethereum consensus to account for this possibility unfortunately opens the door to new attacks based on *ex-ante* reorganizations [15]. Thus, to avoid changes to the consensus protocol, DAS must ideally be completed within four seconds of the corresponding block creation, allowing committee members to attest to block validity and blob data availability simultaneously (named the *tight* fork-choice rule [16]).

Contribution. We demonstrate that DAS can integrate with Ethereum and meet Danksharding's objectives under the tight fork-choice rule. Blob data can be disseminated and sampled within the first four seconds of a consensus slot, allowing nodes to confirm blob and block data correctness at the same time. This removes the need to adapt Ethereum's consensus for delayed availability decisions.

We present PANDAS, a peer-to-peer protocol that supports DAS in Ethereum. PANDAS builds upon the following key features:

- It aligns with recent Ethereum evolutions, including Proposer-Builder Separation (PBS) [24, 33], which introduces powerful builders responsible for preparing block and blob data and ordinary proposers elected by Proof-of-Stake consensus [21]. PANDAS leverages builders for efficient *seeding* of blob data.
- It uses Ethereum nodes to host and sample blob data using peer-to-peer interactions. In contrast to proposals under discussion for improving data availability layers in Ethereum [14, 41], PANDAS employs direct (one-hop) communication, using connectionless networking (UDP) rather than gossip-based broadcast mechanisms available in Ethereum [60]. Interactions adapt to nodes' unavailability and faults, meeting the 4-second deadline in adverse environments or under inconsistent network views by different participants.
- PANDAS supports Ethereum's objectives of decentralization and scalability. Nodes' requirements stay well within the hardware and bandwidth recommendations of Ethereum [6]. Requirements for builders align with the typical capacities of public cloud instances and do not increase with system size.

We implement PANDAS over `libp2p` [5], the network stack of the Ethereum Geth client [4], and deploy 1,000 nodes on an 80-server cluster using representative emulated WAN latencies. Additionally, we utilize a simulator whose results are cross-validated against prototype deployments. This enables us to confidently explore results for up to 20,000 nodes. Our evaluation shows that PANDAS meets the 4-second sampling deadline at all nodes at moderate scales and for the vast majority of nodes at large scales, while maintaining low load on builders and nodes. In contrast, baseline solutions based on GossipSub [60] or the Kademlia DHT [47] do not scale as well, incurring higher overhead and failing to meet the 4-second deadline even at moderate network sizes. Experiments involving a significant fraction of unresponsive nodes and inconsistent views further demonstrate that PANDAS's operations are robust against faults and free-riders, meeting the 4-second deadline for the majority of nodes even when up to 50% of the nodes are misbehaving, and systematically detect data unavailability.

Outline. This paper is organized as follows. We present preliminaries about Ethereum, layer-2 protocols, and PBS (Section 2). We detail the DAS principles and the *Danksharding* objectives and analyze the associated networking and communication requirements (Section 3). We present our model and assumptions, and detail our design objectives (Section 4). PANDAS uses a deterministic assignment of blob data to nodes (Section 5). It operates in three phases, from the *seeding* of blob data by a builder to nodes *consolidation* of this data and its *sampling* (Section 6). PANDAS uses direct and efficient but unreliable UDP communications. An adaptive fetching protocol arbitrates between request redundancy and time constraints (Section 7). We evaluate PANDAS and compare it to baselines (Section 8). We discuss our results (Section 9) before covering related work (Section 10) and concluding (Section 11).

Code availability. Our implementation of PANDAS, its simulator, and reproducibility material are all available open source [51].

2 Preliminaries

We provide an overview of Ethereum, its consensus, the Proposer-Builder Separation principle, and layer-2 protocols.

Ethereum. Ethereum is an open blockchain using Proof-of-Stake (PoS) consensus [21]. Holders of ETH, Ethereum's virtual currency, can lock 32 ETH (their *stake*) or more to operate a *validator*, i.e., a virtual entity participating in the validation of new blocks.

Time in Ethereum is divided into slots of 12 seconds and epochs of 32 slots. In every slot, a new block is added to the blockchain. A subset of validators is deterministically selected to participate in each consensus slot. One of them, the *proposer*, is responsible for forming and spreading a new block. Some validators produce *attestations* of this new block, while others collect these attestations and publish aggregate decisions. As a result, consensus is split into three phases: (1) the broadcast of a new block and its verification by the committee; (2) the propagation and collection of attestations; and (3) the generation and broadcast of aggregate decisions. Each phase accounts for a third of the slot duration, i.e., $12/3 = 4$ seconds.

Servers called full nodes, or simply "nodes" for the rest of this paper, participate in the Ethereum network. Nodes can, but do not have to, host validators. A node is identified by its IP address and a public key, which are shared through *Ethereum Node Records* (ENR) propagated through the network and stored in the underlying Kademlia DHT [42, 47]. While nodes can collect all ENRs by crawling the DHT [11, 19, 55, 59], the association between a node and a specific validator should not be public [34]. Deanonymizing the link between the two leads to security threats such as DDoS or targeted abuse of slashing mechanisms [50]. All nodes, whether they host a validator or not, use the consensus committee's aggregate decisions to determine whether a block is accepted.

The dissemination of new blocks, attestations, and aggregate decisions is supported by GossipSub [60], a peer-to-peer overlay that enables multi-hop, controlled flooding of data.

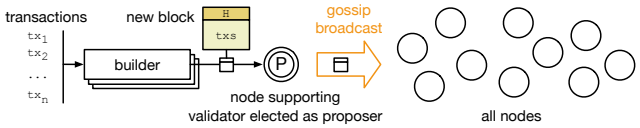


Figure 1: Proposer-Builder Separation (PBS). The proposer, elected based on stake, selects a block among those prepared by builders. The block is broadcast by gossip to all nodes.

Proposer-Builder Separation. Forming new blocks is increasingly computationally expensive, particularly with the rising importance of *Maximal Extractable Value* (MEV) [31]. Any node hosting a validator that can be elected as a proposer would need to provision a powerful server. It is essential to prevent such hardware and bandwidth requirements from leading to a concentration of consensus power among a limited number of actors. Proposer-Builder Separation (PBS), illustrated in Figure 1, addresses this risk by separating the role of *building* a block and the role of *proposing* it for consensus. This enables a few dedicated builders to form new blocks while maintaining decentralized consensus among many nodes that host validators, which only require modest computational and networking capacities, as defined, e.g., by EIP-7870 [6]. With PBS, the node supporting the proposer selects one of the blocks prepared by builders. The block is then broadcast to all nodes using GossipSub. Today, PBS is responsible for around 90% of Ethereum block creation [33, 44], principally through the MEV-Boost network [28]. Builders receive block construction fees for blocks selected by proposers and accepted by consensus; therefore, they are incentivized to produce correct blocks.

Layer-2 protocols. The throughput of Ethereum's chain (i.e., layer-1) is limited to the number of transactions that can fit in a block. Layer-2 protocols move some of the transaction handling and validation processes to a separate layer while benefiting from the security and decentralization of the layer-1 chain. There exist many variants of layer-2 protocols [30, 37].

Rollups are exemplary layer-2 solutions that process transactions off-chain. They publish the resulting state together with a commitment via a call to a smart contract in a regular layer-1 transaction. Furthermore, the compressed transactions are shared through a data availability mechanism. Rollup variants include optimistic ones [25] posting compact hashes of transactions' states, e.g., Arbitrum [39] and Optimism/Bedrock [49], and ZK rollups [18] posting zero-knowledge proofs of validity, e.g., ZkSync [46] or Polygon [52]. The volume of layer-2 transactions that can be *anchored* to the layer-1 chain is directly linked to the volume of blob data that can be made available. This data needs to be available for a sufficient time for a protocol's participants to verify it (e.g., verifying the ZK proof [18, 46, 52] or generating a fraud proof [25, 39, 49]). Unlike regular layer-1 transactions, however, layer-2 data does not need to persist indefinitely nor be verified for correctness by layer-1 nodes.

3 Data Availability Sampling

Ethereum's current mechanism for attaching layer-2 data to layer-1 blocks is EIP-4844 (*Proto-Danksharding*) [10]. It attaches a limited number of data *blobs* (binary objects) to each block. This blob data is broadcast to all nodes. Nodes hosting committee members must

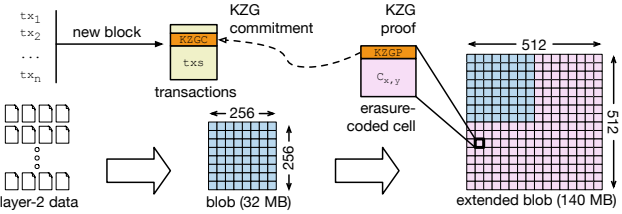


Figure 2: Builder preparatory operations for DAS. 32 MB of data is aggregated in a blob of 256×256 cells, extended to 512×512 cells using erasure coding. Each resulting cell includes a proof (KZGP) linking it with the Kate-Zaverucha-Goldberg commitment (KZGC) in the corresponding blob-carrying transaction.

validate the corresponding commitments contained in blob-carrying transactions. To keep the costs of operating a node reasonable and preserve decentralization, EIP-4844 limits the number of 128-KB blobs to 3 (on average) and 6 (maximum) or to 0.375 and 0.75 MB of data.

The Ethereum roadmap aims for a significant scale-up of the volume of data that can be attached to each block, under the *Danksharding* proposal [23]. This proposal targets up to 32 MB of blob data attached to each block. It is intimately linked to PBS and relies on builders' computational and networking power to collect, aggregate, and share layer-2 data. With such volumes, fully disseminating blob data to all nodes is no longer realistic. Instead, each node receives and stores a subset (shard) of it. For an individual node, receiving a subset does not guarantee the availability of the *complete* blob data. Data Availability Sampling (DAS) is what enables this verification. It consists of three phases. First, the blob is extended using erasure coding. Second, each shard of extended blob data is distributed to a subset of nodes for hosting. Third, nodes collect *samples*, allowing them to consider the data they do not host available (or reconstructable) with an overwhelming probability.

The target parameters are compromises between computational costs for builders, scalability, and supported volume, discussed in the Ethereum community [13, 27]. Figure 2 details the construction of blob data under these parameters. The blob aggregates 32 MB of data, split into cells of 512 B, organized as a 256×256 matrix. Releasing only a subset of blob data is a *data withholding attack*. The base blob is highly amenable to such an attack, as sharing all but one cell makes some data unavailable, threatening the security of layer-2 protocols. To prevent data withholding and allow data reconstruction after losses, the blob is *extended* using a two-dimensional Reed-Solomon erasure code [61]. Each row and column doubles in size but can now be reconstructed from any 50% of its cells. The resulting *extended* blob is now a 512×512-cell matrix. In addition to the 512 B of data, each cell includes a 48 B Kate-Zaverucha-Goldberg proof (KZGP) [40]. This proof links the cell's content to a commitment (KZGC) registered in a layer-1 blob-carrying transaction. In total, the extended blob is $(512 \times 512) \times (512 + 48) = 140$ MB in size including 12 MB of KZGPs.

Following the dissemination of extended blob data, nodes verify its availability by attempting to download randomly chosen cells. Collecting more random samples means higher confidence in the availability or reconstructability of blob data. The number of cells to sample depends on the maximum acceptable rate of false positives, i.e., of incorrectly determining availability. The minimal amount

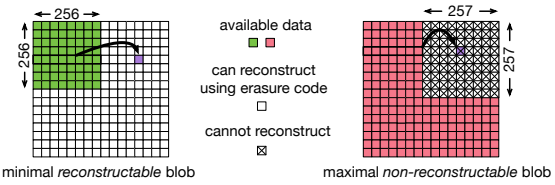


Figure 3: The minimal data enabling reconstruction (left), and the maximal data preventing it (right).

of data necessary to enable reconstruction is half of the cells for either 256 distinct rows or 256 distinct columns, as illustrated by Figure 3-left (note that collecting *any* $256 \times 256 = 65,536$ cells *may not* provide this guarantee). The maximal amount of data that can be shared while preventing reconstruction is the 512×512 matrix minus a 257×257 square sub-matrix, as illustrated by Figure 3-right. If a fraction p of cells was not shared in the network, the probability of not hitting an unavailable cell with s samples is $(1-p)^s$. The false positive probability for availability sampling is, therefore, upper-bounded by $\prod_{i=0}^{s-1} 1 - \frac{257\times257}{512\times512-i}$. Discussions in the Ethereum community [20] suggest using $s = 73$ samples, which gives an upper bound false positive probability lower than 10^{-9}. This corresponds to $73 \times 560\text{B} = 40$ KB worth of samples collected per node.

4 PANDAS: Objectives and Overview

PANDAS is a peer-to-peer protocol that integrates DAS with Ethereum without requiring modifications to its consensus mechanisms. PANDAS targets the parameters proposed by the Ethereum community and listed in the previous section [13, 20, 27]; Its design can easily adapt to variations of these parameters, within reasonable bounds set by the hardware profile of nodes. This section presents our assumptions, details our objectives, and gives an overview of PANDAS.

4.1 Model and Assumptions

This work is based on the following models and assumptions.

System model. The network comprises N nodes. Aligned with Ethereum, the system is open, but each node $n_i \in N$ is identified by an ID i, i.e., a cryptographic hash of its public key. Nodes periodically advertise to store (and refresh) their ENR records in the underlying Kademlia DHT. The ENR of a node contains its ID, public key, and contact information (IP and port). Nodes can be reached directly using this contact information.

The assignment of validators to nodes must remain unknown [34]. Therefore, it must be impossible to distinguish nodes that host validators from those that do not. To maintain decentralization, nodes must have commodity hardware and network capacities.[1]

Dedicated builders propose blocks. For every slot, the elected proposer selects one block from a builder b. Builders have significantly better capacity and connectivity than nodes (e.g., a medium-range cloud instance with a recent multicore CPU and 10 Gbps network upload capacity). The selected builder b is responsible for sending extended blob data to the network of nodes. After that, nodes interact peer-to-peer to exchange this data for retrieval and DAS.

Network views. Each node, including builders, maintains a list of all nodes in the system as its *view* V, i.e., V_b is builder b's knowledge of existing nodes, and V_{n_1} is that of a node n_1. Views are filled by periodically crawling the DHT [19], which typically takes about a minute [11, 55, 59]. Views can be inconsistent (for any two nodes n_1 and n_2, we do not assume that $V_{n_1} = V_{n_2}$, and similarly for builders). They may also be incomplete ($V \cap N \subseteq N$) and contain departed nodes ($V - N \neq \emptyset$). However, thanks to the periodic crawls, views constantly converge towards the actual set of nodes.

Fault/attack model. We consider rational Byzantine behaviors for all participants, i.e., incorrect participants may deviate from the protocol but follow their economic interests (i.e., maximizing gains in terms of rewards while avoiding resource expenditures). Nodes may crash in a fail-silent mode [53] or simply act as freeriders and refuse to respond to some or all incoming requests. Builders aim to obtain block construction rewards while spending as few resources as possible. A selected builder can attempt a *data withholding* attack, i.e., avoid sharing some or all of the blob data to save on operational cost or because it did not produce it. However, under our rational assumption, a builder does not attempt to send *incorrect* data to the network, as doing so would be against its economic interests (i.e, this will be detected when checking KZGP and lead to no rewards, while still incurring bandwidth costs).

4.2 Objectives

The primary objective of PANDAS is to ensure that dissemination and sampling occur within four seconds of creating a block, and that layer-2 clients can easily retrieve blob data. In addition:

- **[Robustness]** Sampling must meet the 4-second deadline even with a large fraction of unresponsive nodes and/or when nodes and builders have inconsistent views.
- **[Scalability]** Timing guarantees must hold with increasing system size, and the load imposed on nodes and builders must remain compatible with hardware profiles recommended for decentralization [2].
- **[Flexibility]** Participating entities may be free to implement local strategies for interacting with other system members, aligning with their financial incentives.

We target the *tight fork-choice* rule [16], i.e., DAS sampling is required before attesting to a block by committee members: a block with valid transactions but unavailable data is attested as invalid. As a result, we do not modify the consensus protocol beyond adding sampling as a verification step for nodes hosting committee member validators. This contrasts with the *trailing fork-choice* rule that postpones sampling to later, and requires non-trivial changes to consensus to be able to revert blocks with unavailable blob data. Similarly, we do not wish to modify Ethereum's discovery protocols (i.e., the DHT holding ENRs) and assume nodes use unmodified crawl mechanisms to collect their views [11, 55, 59].

4.3 PANDAS in a nutshell

The high-level principles of PANDAS are illustrated by Figure 4. At the beginning of a slot, the node hosting the elected proposer selects a block from one of the builders (❶). This block is disseminated via a dedicated, system-wide GossipSub channel (❷). At the same time, the same node requests the builder to publish blob data in the network. The builder *seeds* the network with extended blob cells, using direct communication to nodes in its view (❸). Every node is

[1] EIP-7870 [6] provides guidelines for Ethereum nodes of 4 TB SSD storage, 32 GB memory, and 50 Mbps (download) and 15 Mbps (upload) capacities.

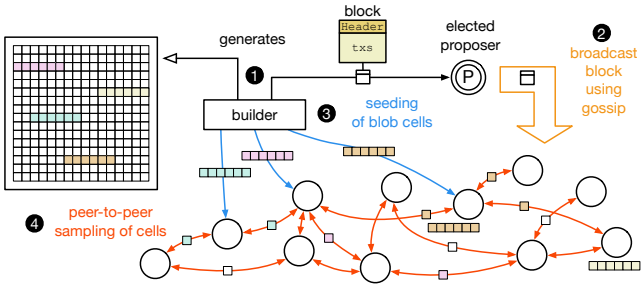


Figure 4: Distributed interactions following the selection of a new block by the proposer (❶). In parallel to the gossip block dissemination (❷), the builder distributes extended blob data to nodes in the network (❸). All nodes interact peer-to-peer to *consolidate* their assignment and collect random samples (❹).

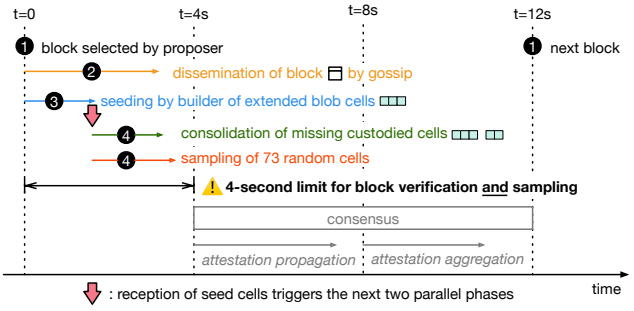


Figure 5: Timeline of events within a slot. Starting from the selection of a new block by the proposer (time 0, ❶), two concurrent processes start for nodes that must both terminate within 4 seconds: block dissemination (❷) and extended blob data dissemination (❸), consolidation, and sampling (❹).

assigned a subset of cells that it must keep in *custody* for the rest of the network. The builder may send only a subset of this assigned data to each node directly. To serve all assigned data, nodes fetch missing cells from other nodes through *consolidation* (❹). In parallel, nodes select 73 cells randomly and send requests to nodes whose responsibility includes them, implementing the *sampling* phase.

Figure 5 represents the timeline of operations. The dissemination and verification of block and blob data are concurrent. Nodes initiate consolidation and sampling when they receive their seed cells from the builder. A node supporting an active validator can vote for a block if the block verification *and* the data availability sampling are successful before the 4-second deadline.

As ENRs do not allow distinguishing between nodes supporting validators and nodes that do not, all correct nodes are expected to custody data as long as they are registered in the DHT. We also assume all correct nodes perform DAS. In particular, we avoid having only committee members performing DAS, as it would reveal the association between validators in the committee and nodes [34].

Communication between all actors in PANDAS is based on one-way UDP networking with no signalling overhead (i.e., there is no establishment of connections or keep-alive messages). We stress that all Ethereum nodes already use UDP in the discovery protocol required to join the network [29]. Blob data is public and, therefore, sent unencrypted, avoiding a time-consuming encrypted channel establishment. Messages are authenticated with a digital signature using the recipient's public key. KZGPs further allow the authenticity of the received blob data to be verified. Peer-to-peer requests may fail silently due to packet loss or incorrect nodes. To alleviate this and meet the deadline, PANDAS relies on builders adopting efficient seeding strategies, reconstructing cells using the erasure code, and nodes employing an adaptive fetching strategy that adapts request redundancy and aggressiveness to the available time budget. Similarly, the impact of freerider nodes that do not participate in custody and consolidation is mitigated by redundancy. Incorrect nodes that forfeit the sampling phase only reduce the system load.

In the following sections, we detail the components of PANDAS. We start with the deterministic association between blob data and nodes (Section 5). Then, we present the three phases of *seeding*, *consolidation*, and *sampling* (Section 6). We finally detail the adaptive fetching strategy (Section 7).

5 Cell to Nodes Assignment

The first component of PANDAS is an assignment between blob data and nodes. A function $\sigma(n_i)$ returns a list of cells from the 512×512 matrix.Node n_i is tasked with their custody, i.e., hosting and serving these cells for sampling queries and access by layer-2 participants.

All nodes and builders know σ. We set two requirements for σ: it must be *deterministic* and *short-lived*. Determinism means $\sigma(n_i)$ must be computed identically by two nodes n_a and n_b even if $V_{n_a} \neq V_{n_b}$.[2] Short-liveness means that the assignment must change periodically and be unpredictable. This prevents the emergence of attacks based on eclipsing nodes in charge of specific cells [45, 62] or censorship of specific data [58].

Even though adjacent cells likely contain data for distinct layer-2 protocols, storing them together on the same node favors efficient reconstruction, as it requires fetching multiple cells from the same row or column. Thus, PANDAS assigns complete rows and columns to each node. The number of rows and columns assigned to each node is a globally known parameter. By default, we use eight distinct rows and columns per node. Each node hosts $8 \times 512 + 8 \times (512 - 2)$ cells, i.e., $8,176 \times (512 + 48) \simeq 4.4$ MB of data per slot.[3]

To enable determinism and short-liveness, the assignment σ is a pseudo-random sortition, as used by Ethereum consensus. For every epoch, a globally verifiable, pseudo-random sortition decides which nodes will be members of committees or proposers in each slot. This decision uses a pseudo-random number generator (PRNG) and an *epoch seed* known one epoch in advance (32 slots, ≃6.4 minutes) from a combination of random values proposed by validators (i.e., the "RANDAO" state [17]). PANDAS builds upon this mechanism by seeding the assignment function σ for an epoch e with its corresponding epoch seed s_e. We extend the definition of σ to include the epoch number, i.e., $\sigma(n_i, e)$ generates eight distinct rows and eight distinct columns for n_i using a PRNG seeded by s_e.

[2] Using consistent hashing, as in DHTs, does not meet this requirement: if n_a knows a node n_c that n_b does not know, n_a may associate cells to n_c that n_b associates to n_a.
[3] Blob data is kept for 4,096 epochs (131,072 slots), thus each node must custody ≃ 559 GB of data, but never more as old data is discarded. In contrast, the size of the layer-1 chain is ≃1.5 TB and will continue to increase. Both fit within the 4 TB of SSD space recommended by EIP-7870 [6]. In the long run, custody of layer-2 data is more economical in terms of disk space than increasing layer-1 block size.

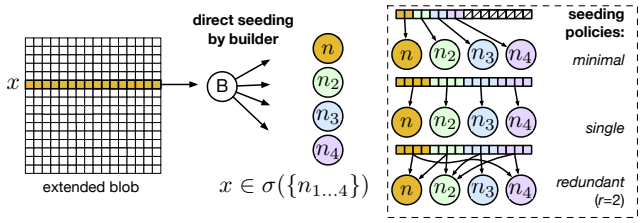


Figure 6: Three seeding policies. The *"minimal"* policy splits the first half of each row or column amongst known peers having it in their assignment. The *"single"* policy splits the entire row or column. The *"redundant"* policy shares each split to r nodes.

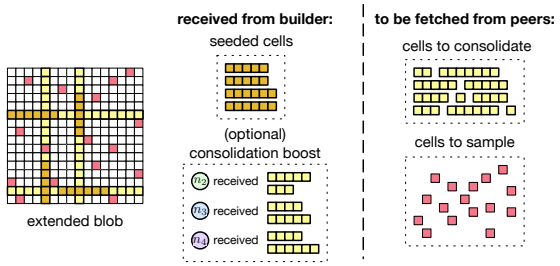


Figure 7: Consolidation and sampling phases must fetch cells from other nodes. Information from the builder consists of initial cells and an optional consolidation boost map.

6 PANDAS Protocol Phases

We detail the PANDAS protocol phases: seeding, consolidation, and sampling, illustrated in Figures 4 & 5. The latter two are concurrent.

6.1 Seeding phase

The interactions start with an initial *seeding* phase. This phase starts when a proposer selects a block from a builder b. In parallel to gossiping the block to the network, the proposer asks b to seed the corresponding blob data to the network. All nodes know the proposer's identity and public key before the slot starts. However, they do not know who b is. Due to the strict time constraints, nodes cannot wait to receive the block via gossip to learn this information and start accepting blob data. To allow nodes to distinguish legitimate blob data, the proposer provides the builder with a digital signature binding b's identity (including its IP address) to the proposer's private key. This signature is attached to every seeding message.[4]

In a naive approach, the selected builder b could send all cells in $\sigma(n, e)$ to every node $n \in V_b$. The necessary outgoing bandwidth now depends on the size of the builders' view, close to or equal to that of the entire network. With ≃4.4 MB per node (eight rows and eight columns) and, say, 10,000 known nodes, the necessary bandwidth budget is 42.9 GB (343.7 Gb). With a 10 Gbps connection, as available with modern, medium-end cloud instances, the process takes more than 30 seconds, largely missing the 4-second deadline.

A better approach is to send a fixed amount of data to the network and determine a level of redundancy for the cells within each row and column. For a row (or column) x, b decides which cells of $x_1, \ldots, x_{512}$ to send to the network and with what degree of redundancy. It dispatches these cells to the nodes assigned to x in the current epoch e that it knows, i.e., $V_b(x) = \{n \in V_b \mid x \in \sigma(n, e)\}$. Each node in $V_b(x)$ receives only a *subset* of its assigned cells. Therefore, each node must still fetch the missing cells from its peers, a process we call *consolidation*, detailed in the following subsection.

Seeding policies. Figure 6 illustrates three example policies. We stress that, according to the flexibility objective of PANDAS, actors should be able to rationally select a strategy based on their economic interest. In the case of the builder, the objective is to maximize the probability of obtaining block rewards (linked to the success of DAS), while minimizing the expenditure of resources.

[4]While the proposer's signature allows verifying the legitimacy of b, the correctness of the received cells' KZGP from b cannot be checked against the KZGC before receiving the block and its blob-carrying transactions. It is, however, not in the builder's interest to send fake blob data that will eventually cause it to lose the rewards.

A *"minimal"* policy sends a single copy of *half* of the cells of x, i.e., $x_1, \ldots, x_{256}$ (i.e., the minimal amount of data necessary to reconstruct the row or column). The builder splits $x_1, \ldots, x_{256}$ into $|V_b(x)|$ parcels of adjacent cells and distributes them randomly to up to 256 nodes in $V_b(x)$. It repeats the process for all rows and columns. The total amount of data sent out is $256 \times 256 \times (512 + 48) = 35$ MB. This strategy fails to make data available if even a single message is lost. We primarily use it as a baseline for the builders' costs.

A second, *"single"* policy leverages the redundancy allowed by the erasure code. It operates similarly to the minimal policy but sends a single copy of *all* of the cells of each row or column x, split to up to 512 nodes in $V_b(x)$. In total, it sends out the size of the extended blob, i.e., 140 MB. This strategy's rationale is that even if half of the cells are lost, nodes can still reconstruct the row or column using the erasure code.

The third, *"redundant"* strategy, adds further redundancy by sending r copies of each cell. It starts from the single policy, splitting the cells of x between nodes in $V_b(x)$. Then, each parcel is further assigned to $r - 1$ randomly selected distinct nodes in $V_b(x)$. We use $r = 8$ by default. The outgoing bandwidth usage for the builder is 1,120 MB = 1.09 GB.

6.2 Consolidation phase

The objective of the second phase, consolidation, is for a node n to rapidly get hold of all the cells of rows and columns assigned by $\sigma(n, e)$. Thanks to the erasure code, collecting half of the cells of a given row or column is sufficient to consolidate it.

Consolidation begins at n upon receipt of seed cells from builder b. If n receives a request from another node linked to a slot for which it has not yet received its seed cells, it activates a timer (we use a default value of 400 ms). Consolidation starts without seed data if the timer expires before n receives cells from b, either due to packet loss or because b does not yet know n (i.e., $n \notin V_b$).

The orchestration and timing of requests for fetching missing cells are delegated to PANDAS's fetching strategy, shared with the sampling phase, and detailed in the next section.

To fetch missing cells, n contacts peers with overlapping rows and columns, $C_n(x) = \{n' \in V_n \mid x \in \sigma(n, e) \wedge x \in \sigma(n', e)\}$. In a view with 10,000 nodes assigned eight rows and eight columns each, each row or column is assigned to $\frac{10000 \times (8+8)}{512 \times 2} \simeq 156$ nodes on average. Depending on the builder's seeding strategy, each may have received only a subset of the data. Asking many peers for cells may increase the chances of "hitting" the ones that received the needed cells via seeding, but it leads to many duplicates. In contrast, asking

only a few random peers may require the selected nodes to finish their consolidation to respond, leading to a lengthy response delay.

A fast and effective consolidation aligns with the economic interests of the builder. It improves the odds that sampling finishes on time, and the builder may send less data to the network. We improve these two factors with *consolidation boosting*, as illustrated by Figure 7. The builder b attaches to the seeding message to n a map CB. For every row and column $x \in \sigma(n, e)$, $CB(x)$ lists the cells received by other nodes $n' \in V_b$ where $x \in \sigma(n', e)$. The consolidation boosting map CB enables n to identify which nodes are likely to receive specific cells more quickly and prioritize them for requests.

6.3 Sampling phase

The third phase of PANDAS is the *sampling* phase. It starts at the same time as consolidation and takes place concurrently.

Node n randomly selects 73 cells to sample. This selection must be unpredictable (i.e., unlike σ). For every sample, n can determine the nodes hosting an intersecting row or column in V_n. In a 10,000-node network, 156 nodes on average can have a copy of a given cell. The selection of targets for sampling and the orchestration and scheduling of requests are delegated to PANDAS's fetching algorithm that we describe next.

7 Adaptive Fetching

Both consolidation and sampling require fetching cells from other nodes. We detail an adaptive fetching algorithm that handles both collections simultaneously, and recommend default parameters.

The fetching algorithm inputs a set of cell identifiers, as illustrated by Figure 7. In addition, it may receive a consolidation boost map CB. The algorithm aims to retrieve all cells before the 4 s deadline.

Target nodes are identified by a node n from its view V_n using the assignment σ. Some of these nodes may be offline or unresponsive. As PANDAS uses connectionless communications using UDP, the network may silently lose queries or response messages. Sending queries for cells sequentially bears the risk of missing the deadline. On the contrary, sending queries to multiple nodes that hold a copy of each desired cell generates a swarm of messages in the network. This leads to congestion risks, suggesting the need for a compromise between cautious fetching initially and a more aggressive approach with more redundant queries as the deadline approaches.

Consolidation processes at different nodes are executed concurrently. A queried node n_i may have a cell c in its assignment $\sigma(n_i, e)$ but have not yet received it from the builder or through consolidation. Nodes receiving a query for assigned cells they do not yet have buffer this query and respond when they can (if the cells are never received, they never respond, i.e., there is no negative acknowledgment). Thus, a querying node may allow sufficient slack time for queried nodes to respond, particularly when time is early in the slot.

Fetching algorithm. Fetching operates in rounds, as illustrated in Figure 8. It adapts query redundancy and timeouts as time progresses and the deadline nears. For this purpose, each round i is associated with a timeout t_i and a redundancy factor k_i. Algorithm 1 details the process at a node n. The FETCH procedure receives a set of cells to fetch F and an optional consolidation boost map CB (line 1). Node n considers as *queryable* nodes all of its view V_n upon the initial call to FETCH, saved as a working copy Q (line 2). Any node in Q

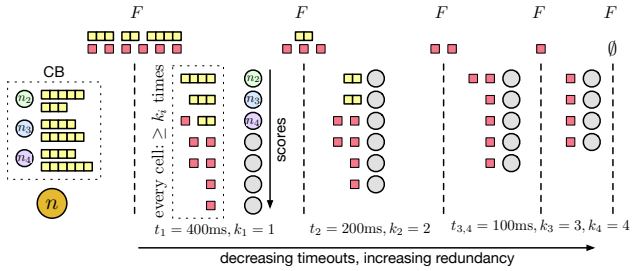


Figure 8: Node n determines a set of nodes to query at each round, adapting query redundancy and timeouts.

Algorithm 1 Adaptive fetching at node n in epoch e.

```
1:  procedure FETCH(F, CB)
2:      Q ← V_n; i ← 1                                  # Queryable nodes and round number
3:      while F ≠ ∅ ∧ i < i_max do                       # Until all fetched or too many rounds
4:          for each q ∈ Q do                           # Assign scores to queryable nodes
5:              q_cells = {σ(q, e) ⊆ F}                 # Cells of interest ...
6:              q_score = ‖q_cells‖                     # ... score is number of cells
7:              if CB_q ≠ ∅ then                        # If node in consolidation boost map ...
8:                  q_score ← q_score + (|F ∩ CB_q|) × cb_boost
9:                                                      # ... boost score for each cell received by seeding
10:         sort Q by decreasing node score as q_1, ..., q_|Q|
11:         P = ∅; U = F; j = 1                         # Query Plan and cells Under redundancy
12:         while U ≠ ∅ ∧ j ≤ |Q| do                    # Should/can plan more queries
13:             if (q_j.cells ∩ U) ≠ ∅ then             # At least one cell of interest
14:                 P ← P ∪ (q_j, q_j.cells ∩ U)        # Plan query
15:                 U ← {c ∈ F | |{p ∈ P | c ∈ p.cells}| < k_i}
16:                                                     # Update set of cells with insufficient redundancy
17:             j ← j + 1                               # Consider next node in sorted Q
18:         for each p in P do                          # Send out queries from the query plan
19:             QUERYCELLS(p.node,p.cells)              # UDP async. query
20:             Q ← Q \ p.node                          # Nodes are queried only once
21:         SLEEP(t_i); i ← i + 1                       # Wait before next round
22:     return (F ≠ ∅)                                  # Success if all cells fetched within round limit
23: procedure UPONRECEIVE(C)                            # Receiving a set of cells C
24:     F ← F \ C                                       # Receive new cells
25:     while ∃ row or column x with [256 : 512) cells do   # Can use code
26:         x ← RECONSTRUCT(x,R)                        # Reconstruct full row/column
27:         F ← F \ {c ∈ x}                             # No need to fetch reconstructed cells
```

will be queried at most once. The fetching process is in three steps: *scoring*, *planning*, and *execution*.

In the scoring step (lines 4 to 10), queryable nodes in Q are assigned a score, i.e., the number of their assigned cells still missing for n (lines 5 and 6). If a consolidation boost was received, nodes are given a score boost of *cb_boost* for each cell declared as seeded by the builder and missing from F (lines 7 and 9). The set of queryable nodes is then sorted by decreasing score values (line 10).

The planning step (lines 11 to 17) prepares the set of queries as a set P. Each planned query $p \in P$ is associated with a node p.node and queried cells p.cells. Each missing cell from F must be queried from k_i nodes. Starting from the node with the highest score, q_1, the step greedily selects nodes with cells of interest as long as this criterion is not met. For this, it maintains a set U listing the cells for which insufficient redundancy currently exists in P. A node q_j is planned to be queried for cells with insufficient redundancy (line 14), before updating U (line 15).

Finally, the execution step sends out the queries asynchronously (lines 18 to 20) before waiting for t_i ms before the next round. A queried node is removed from Q and is not used again. Upon correct reception by the target node, the handler either responds with the queried cells if all are available or buffers the query for a delayed reply. The response is received by the UPONRECEIVE function as

a set of cells C (lines 23 to 27).[5] When receiving new cells, the algorithm checks if an incomplete row or column now contains 256 or more cells (line 25) and, if so, reconstructs them (line 26).

Default parameters. The fetching algorithm is primarily parameterized by the round durations and query redundancy vectors t and k, as well as the score boost for consolidation *cb_boost*. We use the following universal parameters, but stress that nodes could select them differently, e.g., based on local connectivity.

In the first round, $i = 1$, the strategy aims to maximize the number of cells received (and reconstructed) using as few messages as possible (i.e., $k_1 = 1$). We use a duration of $t_1 = 400ms$ based on estimated time for the builder to send out initial cells and on inter-node latencies, as we will detail in Section 8. In subsequent rounds, we reduce this time by half but no lower than $100ms$, i.e., $t_2 = 200ms$ and $\forall j \geq 3, t_j = 100ms$ (up to t_{50}). Similarly, we increase the aggressiveness of queries by increasing the redundancy factor by two every round until a maximum of 10, i.e., $r_2 = 2, r_3 = 4, \ldots, \forall j \geq 6, r_j = 10$. Finally, we set *cb_boost* $= 10,000$ to give an overwhelming advantage to nodes with seeded cells of interest.

8 Evaluation

We structure our evaluation around the following claims:

- **C1:** PANDAS completes DAS within 4 s and supports the tight-fork choice rule under Danksharding requirements.
- **C2:** PANDAS bandwidth requirements for nodes are below Ethereum suggestions for decentralization (50/15 Mbps download and upload per EIP-7870 [6]) and, for builders, below typical cloud offerings (10 Gbps upload).
- **C3:** PANDAS satisfies **C1** even under a high percentage of free-riding or failed nodes and with highly inconsistent views.
- **C4:** PANDAS satisfies **C1**–**C3** scaling up to 20,000 nodes.
- **C5:** Relying on existing peer-to-peer overlays (GossipSub [60] and Kademlia [47]) for DAS does not allow satisfying **C1**.

PANDAS [51] is implemented in Go, extending `libp2p` [5], the network stack of the Ethereum Geth client [4]. Block dissemination relies on `libp2p`'s GossipSub implementation.

We aim to evaluate PANDAS in a real-world environment and verify its scalability in large networks. Achieving both objectives with the PANDAS prototype would require a prohibitive amount of resources. We thus opt for a hybrid approach. We deploy 1,000 instances of PANDAS in a cluster, emulating representative WAN latencies. To evaluate PANDAS up to 20,000 nodes, we use a simulator, whose accuracy is validated against the deployment results.

8.1 Prototype deployments

We run 1,000 PANDAS instances on a cluster of 80 servers, each equipped with an 18-core Intel Xeon Gold 5220 CPU and 96 GB of RAM. This level of consolidation (13 instances per server) was selected through careful load testing to avoid CPU contention and increased latencies compared to non-consolidated deployments.

Network emulation. We use network emulation using `tc` to reproduce WAN settings. There is no publicly available data on node-to-node latencies in the Ethereum network. However, a recent large-scale measurement campaign [43] has collected all-pair latencies in IPFS [9], a planetary-scale storage system that shares the scale and decentralization objectives of Ethereum. We use this trace for our network emulation. Round-trip latencies range from 8 ms to 438 ms with an average of 64 ms. The topology contains 10,000 vertices, to which we assign nodes randomly. We limit each node connection to 25 Mbps. We deploy a builder as a dedicated server, with a connection capped to 10 Gbps, assigning it to a vertex in the topology randomly selected among the 20% with the best average latency to all other nodes, i.e., nodes likely deployed in a cloud. UDP communication in the cluster is subject to a packet loss rate of 3%, according to our observations.

Evaluation metrics. Our primary metric of interest is the distribution of completion times for PANDAS's three phases, from the moment the builder is selected. The *time to seeding* is when a node has received its initial seed data. *Time to consolidation* and *time to sampling* refer to the periods when a node has received (or can reconstruct) its assigned eight rows and columns, and its 73 random cells, respectively. Additionally, we monitor the bandwidth costs and the number of messages for all nodes and the builder. We consider a fault-free scenario in this section, where all nodes participate in the protocol and have a complete view of the system (i.e., $\forall n \in N, V_n = N$). For all experiments, we present distributions over 10 slots (i.e., 10 cycles of seeding, consolidation, and sampling).

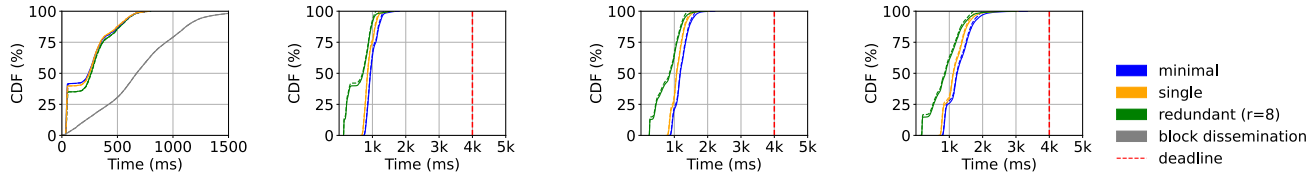
Phases timing. Figure 9 presents the distributions of times to seeding, consolidation, and sampling. We consider the three seeding strategies of Section 6.1: *minimal*, *single*, and *redundant* with $r = 8$. Only solid lines are of interest in this section; dashed ones represent simulator results that we will discuss in the next section. We illustrate the distribution of the reception time of the block via a global GossipSub channel (initiated by a randomly chosen node serving as the proposer), for comparison purposes, in Figure 9a.

We observe that the time to seeding is similar for the three strategies, as our builder's available bandwidth is not a bottleneck (the amount of data sent out is 36.6 MB, 149 MB, and 1,208 MB, respectively, for the three strategies). We observe an impact on the tail of the distributions: the maximum time to seeding is 700, 819, or 936 ms, respectively, for the three strategies (99th percentiles, or P99, are 698, 705, and 715 ms). The "step" around 64 ms corresponds to nodes assigned to well-connected vertices in the emulated topology, which are typically nodes deployed in the same cloud and/or region as the builder. Overall, all nodes receive their seed cells before the end of the first second of the slot.

We present the time to consolidation both from the reception of the seed data by a node (Figure 9b) and from the beginning of the slot (Figure 9c). The builder provides the consolidation boosting map to the nodes. We can observe the impact of the builder's seeding strategy. The minimal strategy results in a consolidation taking up to 2,213 ms (P99=1,756 ms) from the reception of the seed data, and the single strategy has a maximum time of 2,046 ms (P99=1,595 ms). In contrast, the redundant strategy reduces this time to 1,985 ms (P99=1,558 ms). Median times to consolidation for the minimal, single, and redundant strategies (from the beginning of the slot) are 1,178 ms, 1,072 ms, and 869 ms, respectively.

The time to sampling distribution, our primary metric of interest, is given by Figure 9d. This distribution depends on the builder's

[5] For clarity, we omit in Algorithm 1 the verification checks performed when receiving C (e.g., verifying the cells KZMPs if/when the block header is available).



(a) Seeding (from start) **(b) Consolidation (from seeding)** **(c) Consolidation (from start)** **(d) Sampling (from start)**

Figure 9: Distribution of times for the three phases of PANDAS across all nodes, for the three seeding strategies. All times are from the start of the slot, except for Figure 9b where time is counted from the reception of the seed cells (as shown by Figure 9a).

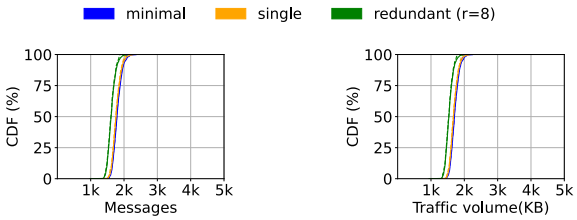
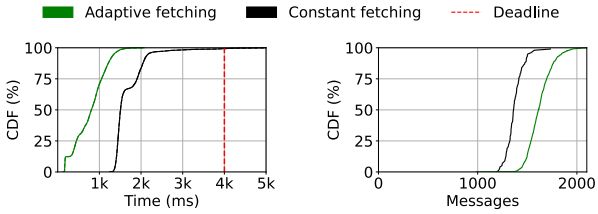


Figure 10: Distribution of messages and traffic volume for fetching across nodes, for different seeding strategies.

Round	1	2	3	4
Messages sent	341 ± 20	261 ± 58	185 ± 35	113 ± 22
Cells requested	4174 ± 100	2426 ± 96	923 ± 63	294 ± 40
Replies received in round	228 ± 22	143 ± 14	120 ± 20	69 ± 25
Replies received after round	107 ± 39	114 ± 25	56 ± 20	61 ± 3
Cells received in round	2420 ± 180	949 ± 170	535 ± 82	191 ± 22
Cells received after round	1128 ± 113	1478 ± 91	383 ± 52	23 ± 8
Received cells duplicates	0 ± 0	187 ± 42	142 ± 29	64 ± 12
Cells reconstructed	615 ± 126	566 ± 90	86 ± 29	32 ± 17
Cumulative coverage of F	56%	81%	96%	99%

Table 1: Fetching algorithm performance in successive rounds (values averaged over all nodes, ± is the standard deviation).



(a) Time to sampling **(b) Messages in and out**

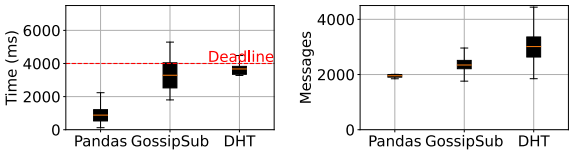
Figure 11: Comparison of the performance of adaptive fetching, as used by PANDAS, and a non-adaptive approach.

seeding strategy, which impacts the time to consolidation. The minimal strategy results in a maximum of 3,341 ms (P99=2,303 ms); still, 100% of the nodes fetch their samples by the deadline. The single strategy also meets the deadline, with a maximum delay of 3,062 ms (P99=2,068 ms). Finally, the redundant strategy matches the deadline safely for all nodes, with a maximum of 3,009 ms (P99=2,020 ms). The median times to sampling are, respectively, 1,235 ms, 1,122 ms, and 882 ms. The reduction in sampling times with increased availability of seed cells (via increased redundancy) is due to reduced contention on peer bandwidth, which in turn speeds up the fetching operation. We observe, however, that if the block dissemination latency (Figure 9a) were to be added to these times, meeting the 4 s deadline would be at risk for many nodes, even with the redundant strategy. This confirms our claim that DAS must start *concurrently* to block dissemination if we are to integrate it with consensus under the tight fork-choice rule.

Bandwidth consumption. Figure 10 presents the distribution of the number of messages used by nodes in the fetching phase, and the corresponding traffic volume, summed for both directions. The redundant seeding strategy results in fewer messages exchanged between nodes and, as a result, lower bandwidth requirements. This is because more nodes already hold the requested cells, which reduces the need for retries (i.e., fewer rounds) during consolidation and sampling. Even with the single seeding strategy, which only marginally differs from the minimal one, the requirements are far below EIP-7870 recommendations; the maximum traffic volumes are 2.26, 2, and 1.99 MB for the three strategies.

Fetching analysis. Table 1 presents an analysis of the progress of fetching for the first four rounds. All values discussed in this paragraph are averages over the 1,000 nodes, together with the standard deviation. We use the redundant seeding strategy, and nodes receive 2420 cells (± 180). Starting from 4,174, the number of requested cells decreases as the coverage of F (i.e., set of cells to fetch) increases, either through reception or reconstruction (e.g., 615 reconstructed cells in the first round). We distinguish between replies received *in* a round i, i.e., before the timeout t_i expires, and after. The latter case leads to redundant requests but illustrates the tradeoff between caution and eagerness Algorithm 1 implements. A majority of requests result in replies before the timeout, and a majority of cells are received on time in the round. Receptions after the round generally occur with a significant delay; adjusting timeouts to account for such tail latencies leads to lower success rates. While Table 1 only shows the first four rounds—after which 99% of the nodes have completed fetching—the process requires up to 6 rounds for the slowest nodes (P90=3, P99=4).

Impact of adaptive fetching. We evaluate in Figure 11 the impact of adaptive fetching. We consider the redundant seeding policy, i.e., the green distribution in Figure 11a is the same as in Figure 9d. For comparison, we employ a *constant* fetching strategy, which utilizes a fixed timeout for all rounds ($t = 400$ ms) and a fixed redundancy ($k = 1$), as represented in black. The constant strategy uses fewer messages, as it asks only a minimum of one node for each missing cell in each round, and leaves more time for nodes to respond. However, it drastically impacts the time to sampling, resulting in a maximum of 4,129 ms (P99=3,513 ms, median=1,546 ms), and some nodes miss the deadline. This illustrates the interest of dynamically



(a) Time to sampling (b) Messages

Figure 12: Distribution of time to sampling and messages compared to baselines based on GossipSub and the Kademlia DHT.

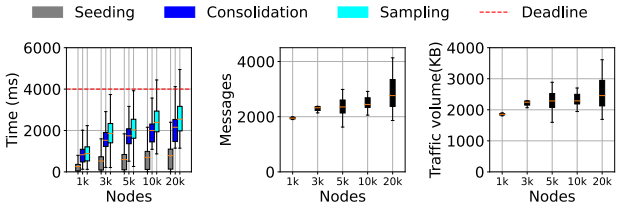
adapting aggressiveness and redundancy to cope with the tight time constraints imposed by the tight fork-choice rule.

Comparison to baselines using existing P2P layers. We finally compare our approach to two alternative methods based on the use of existing peer-to-peer protocols available in `libp2p`.

Some proposals for data availability support in Ethereum [14, 41, 56], that we will further discuss in Section 10, suggest using GossipSub [60] for the partial dissemination of blob data. We instantiate this idea by having all nodes subscribe to GossipSub channels corresponding to each unit of custody—that is, each group of eight rows and eight columns as assigned by σ. We disable explicit consolidation, but instead rely on GossipSub's gossiping within each channel to disseminate the assigned cells, and use the same sampling phase as in PANDAS. Therefore, the main difference is that the dissemination of seed cells occurs through peer-to-peer gossip within each channel, rather than through direct seeding by the builder as in PANDAS. In this 1,000-node network, each GossipSub channel involves approximately ~ 16 nodes assigned to the corresponding unit of custody. The builder sends $r = 8$ copies of each unit of custody to the nodes in the corresponding GossipSub channel, which is configured with the default fanout of eight peers. As a result, the builder's outgoing traffic volume is the same as in the redundant seeding strategy of PANDAS, i.e., eight times the total blob size.

Another possible approach [12] is to use the Kademlia DHT [47] for storing and retrieving cells using multi-hop routing. We implement it by mapping rows and columns to one dimension and splitting it into *parcels* of 64 adjacent cells. Parcels are then stored in the DHT by the builder using the `put(key)` operation. To ensure a fair comparison with PANDAS and the GossipSub baseline, the builder performs eight `put(key)` operations per parcel, storing it at each of the eight closest peers to the hash of the parcel's contents—therefore, the builder uses the same total bandwidth as in the other approaches. Nodes are responsible for the range of keys (and, therefore, parcels) assigned by the DHT, and we disallow consolidation. Sampling uses `get(key)` operations to fetch necessary parcels.

Figure 12 shows the distribution of time to sampling for PANDAS using the redundant seeding strategy ($r = 8$) and for the two baselines, as well as the distribution of the number of messages. With 1,000 nodes, 24% of GossipSub nodes and 17% of DHT nodes fail to complete sampling within the 4 s deadline. The average sampling delay for GossipSub nodes is 3,660 ms (P99=3342 ms), while PANDAS nodes complete sampling significantly faster, i.e., on average in 882 ms (P99=1935 ms), with all nodes completing well within the 4 s deadline. In terms of messaging, the DHT and GossipSub baselines incur significantly higher overhead in the number of messages compared to PANDAS. On average, PANDAS, GossipSub, and DHT nodes send 1,613, 2,370, and 3,021 messages. For the DHT baseline, the messaging overhead of storing and retrieving parcels is especially high due to multi-hop routing (i.e., DHT traversal).



(a) Time (b) Fetching messages (c) Fetching bandwidth

Figure 13: Simulation of seeding, consolidation, and sampling times for PANDAS with a various number of nodes.

8.2 Large-scale simulations

In addition to the prototype deployment detailed in the previous subsection, we implement PANDAS protocols in PeerSim [48], a Java simulator for large-scale evaluation of peer-to-peer systems [51]. This implementation closely follows the one over `libp2p`. We also implement the two baselines detailed above.

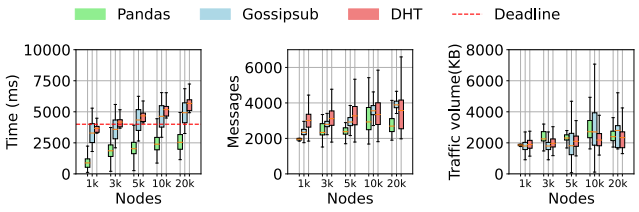
We simulate the same latency trace as for the deployment.[6] We also enforce a fixed 3% loss rate for UDP packets as experienced in the testbed. When running on a server with 256 GB of memory, the simulator can scale up to 20,000 nodes.

Simulator validation. First, we validate that the simulator results match those of the deployments. In all plots of Section 8.1, dashed lines report the results obtained with 1,000 simulated nodes. In all cases, the two lines are (almost) indistinguishable. Our evaluations at smaller scales (not shown) have the same property. The validation of simulation results at moderate scales gives us confidence in the simulator's ability to provide accurate results at higher scales.

Scaling. We investigate PANDAS's scalability from 1,000 to 20,000 nodes. Figure 13 presents the distribution of times to seeding, consolidation, and sampling using the redundant seeding strategy (Figure 13a) and the corresponding messages (Figure 13b) and bandwidth (Figure 13c). With 10,000 nodes, the current scale of the Ethereum network [3], all nodes successfully sample before the 4 s deadline. With 20,000 nodes, 10% fail to meet the deadline, mapping to nodes with poor simulated connectivity (connected from remote area of the geo-distributed network). We identify the scattering of seed data and the cost of consolidation as the primary reasons, as nodes have to contact more peers to collect their rows and samples, and take more time before being able to answer sampling requests. Nodes located in clouds do not suffer from significantly higher times, highlighting the need to host validator-supporting nodes in well-connected infrastructure.

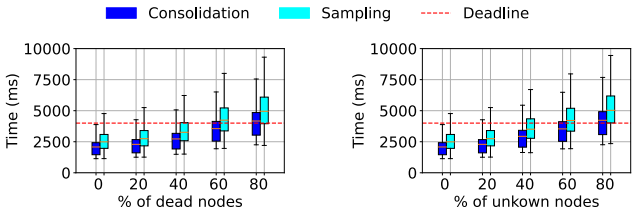
The impact of the increasing scattering of seed cells with larger network sizes is also reflected in Figure 13b and Figure 13c, which show the number of messages and traffic volumes for fetching cells during consolidation and sampling. The average number of messages per node for networks of 1K, 3K, 5K, 10K, and 20K nodes is 1,956, 2,231, 2,247, 2,291, and 2,443, respectively. The corresponding peak traffic volumes are 1.9, 2.1, 2.2, 2.2, and 2.4 MB over the entire slot. We observe that even in the most demanding scenario with 20K

[6] When using more than 10,000 nodes, we reuse vertices randomly for the assignment.



(a) Time to sampling **(b) Messages** **(c) Bandwidth**

Figure 14: Simulation of blob dissemination time for PANDAS and the two baselines, for various number of nodes.



(a) Dead nodes **(b) Out-of-view nodes**

Figure 15: Simulation of time to consolidation and time to sampling for increasing numbers of dead and out-of-view nodes, in a 10,000-node network.

nodes, the maximum traffic volume is transmitted in approximately 2.2 seconds, keeping the average bandwidth requirement well within the EIP-7870 recommendations [6].

Comparison to baselines. Figure 14 compares PANDAS to the two baselines in scales up to 20,000 nodes. Results for 1,000 nodes are consistent with the ones for testbed deployment reported in Figure 12. While the GossipSub-based baseline meets the deadline for a majority of nodes with 1,000 nodes, it fails to do so starting with 5,000 nodes. However, it plateaus for higher node counts, as GossipSub topics become more efficient with a higher number of participants. The DHT-based baseline is unable to meet deadlines for most nodes at all scales and shows linearly increasing times to sampling for increased system sizes. For both systems, the gap to PANDAS in terms of time-to-sampling latency widens as the system size grows. The number of messages is also significantly higher for the baselines than for PANDAS, with important variability for the GossipSub-based baseline as the system size increases.

Behavior under faults. We evaluate PANDAS's robustness under two types of faults: *dead nodes* and *out-of-view nodes*.

In the *dead nodes* scenario, a fraction of nodes is assumed to have crashed and do not respond to any messages. This also corresponds to the case of *free-riding* nodes, which do not wish to spend upload bandwidth to answer requests. The builder and the remaining correct nodes are unaware of these failures. Therefore, the builder seeds data to all nodes, including the dead ones, and includes them in consolidation boost maps. As a result, some seed cells are lost, and correct nodes may attempt to contact dead nodes during fetching, leading to timeouts and retries. On the other hand, in the *out-of-view nodes* scenario, all nodes are correct and receive their assigned seed cells from the builder. However, each node only has an incomplete view of the network, and these views are not consistent. For example, if 20% of nodes are out of view, each node is only aware of a randomly chosen 80% of the full node set. This affects both consolidation and sampling, as requests may fail if the sender lacks knowledge of a suitable peer.

In Figure 15, we vary the proportion of dead or out-of-view nodes from 0% to 80% (in 20% increments) and measure the impact on both time to consolidation and sampling. The network size is 10,000 nodes. We observe that for 0%, 20%, 40%, 60%, and 80% of *dead nodes*, 92%, 83%, 74%, 45%, and 27% of nodes complete sampling within the 4-second deadline, respectively. In the case of *out-of-view nodes*, for 0%, 20%, 40%, 60%, and 80% of nodes being out of view, 92%, 83%, 67%, 47%, and 25% of nodes complete sampling within the deadline, respectively.

In both scenarios, we observe that beyond 50% dead or out-of-view nodes, over half of the correct nodes fail to meet the deadline, which would prevent consensus from being reached and cause the blockchain to stall. These scenarios are, however, unlikely in practice and would impact other key mechanisms, such as block dissemination, thereby preventing consensus from succeeding anyway. A more likely scenario of temporary latency spikes could delay sampling completion for a small fraction of nodes in a given slot but would not cause persistent unavailability. Lagging nodes can perform multiple rounds of sample fetching per 12s slot, enabling them to catch up once network conditions stabilize.

Summary. Our evaluation using a prototype deployment and large-scale simulations of PANDAS and two baselines confirms our claims. PANDAS supports DAS within 4 s for all nodes (up to 10,000) and the vast majority of them (for 20,000), and enables the tight fork-choice rule (**C1** and **C4**). The bandwidth requirements for builders and nodes are below Ethereum recommendations and compatible with its decentralization objectives under the PBS principles (**C2** and **C4**). The evaluation of fault scenarios shows that PANDAS supports these claims with a significant fraction of failed or out-of-view nodes (**C3** and **C4**). In contrast, the two baselines fail to meet these criteria, especially as the system size increases (**C5**).

9 Discussion

We discuss PANDAS and classical decentralized systems concerns.

Sybils. A node in Ethereum, and thus PANDAS, does not have to support validators to participate in peer-to-peer interactions, e.g., block dissemination. This opens possibilities for *Sybil* attacks, where an attacker operates multiple nodes to bias the system operation.

Sybils can perform *general* attacks, where they join the DHT and GossipSub channels and stop answering queries or forwarding data, disrupting the system merely by their overwhelming presence. Our evaluation shows that PANDAS is robust against many nodes that ignore sampling and consolidation requests, provided the builder uses sufficient redundancy in its seeding strategy. Proposals for increasing IP diversity in Ethereum's discovery mechanisms [42] could strengthen this robustness.

A *targeted* use of Sybils consists of carefully placing them in the peer-to-peer network to prevent specific nodes from interacting with it (an *Eclipse* attack) or to censor specific information [35, 45, 58, 62]. PANDAS makes the network fully connected and randomizes exchanges, making Eclipse attacks irrelevant. This contrasts with designs based on GossipSub trees, where an attacker could position its Sybils as the first neighbors of the builder and disrupt the early

dissemination of blob data. Another targeted attack scenario targets specific *content*. In PANDAS, disrupting the sharing of specific blob data would require (1) knowing before blob data dissemination which cell will contain such data and (2) positioning Sybil nodes in the network to make the corresponding row and column difficult to reconstruct (i.e., disallow fetching half of its cells). Condition (1) does not hold as the cell location is known only upon reception of the block. Condition (2) would require the attacker to generate enough identities to control the corresponding row and column, which is highly improbable considering that σ changes unpredictably every 6.5 minutes, less time than what ENR crawling requires.

Limiting openness. As one of PANDAS goals is to avoid modification to Ethereum core mechanisms, it follows its open-network design. An alternative design could limit participation to validator-holding nodes and restrict other nodes to being only observers. This would drastically reduce any potential risk associated with Sybil attacks, as an attacker can only generate one identity for every 32 ETH they hold. It would, however, limit decentralization by switching to a semi-permissioned system, where only stakeholders can participate. *Proof-of-validator* [38] is an anonymous credential scheme based on zero-knowledge proofs (ZKP). It could enable this limitation if integrated with node discovery, i.e., the crawl of the DHT for ENRs. This approach would come at a significant complexity cost, even for an observer (i.e., gathering the list of validators and verifying a ZKP for every crawled ENR).

Handling free riders. Nodes using the system while contributing minimal resources, or free riders, are unavoidable in decentralized systems [36]. We evaluated that PANDAS is robust to a large fraction of free-riding nodes in Section 8.2. In Ethereum, block and blob building, proposal, and validation are incentivized through monetary rewards; however, there is no incentive for interactions within the peer-to-peer network (e.g., for correctly answering ENR discovery requests in the DHT). Similarly, PANDAS does not have incentives for nodes to participate in the consolidation, sampling, and hosting of blob data. Also departing from our objective of no modification to Ethereum, a possible direction to include incentives for DAS operations would be to extend the above-mentioned *proof-of-validator* mechanisms with *proof-of-custody* incentives [26].

10 Related Work

We begin by outlining alternative proposals for implementing data availability layers in Ethereum. Then, we explore the broader history and foundational literature on data availability sampling.

Alternative data availability layers. The Ethereum community has mainly explored gossip-based mechanisms for data availability. PeerDAS [56], scheduled for deployment in the forthcoming Fusaka fork, applies one-dimensional (row-wise) erasure coding and introduces column-based sampling across 128 GossipSub subnets, with each subnet responsible for a single column—that is, a vertical slice across all blobs in the block. Validators are deterministically (and statically) assigned to subnets and verify the availability of their assigned columns when a block is proposed. Because sampling relies on GossipSub dissemination, PeerDAS can only support small blob sizes (up to six 128 KB blobs) and cannot scale to the 32 MB blobs envisioned for full Danksharding. SubnetDAS [14], in contrast, is only a conceptual proposal outlined in a research forum post. It extends PeerDAS with two-dimensional coding but replaces column sampling with row-and-column assignments: each validator attests availability solely on the basis of receiving its designated row and column. This removes random sampling from the consensus path, which weakens security guarantees, as validators may attest positively even if large portions of the blob remain unavailable.

Alternative network-layer mechanisms for DAS have also been evaluated. A recent study [12] highlights the inefficiencies of using the Kademlia DHT [47] for DAS, particularly the overhead of seeding cells to nodes involving traversing the DHT.

Data availability sampling. The idea of verifying data availability by sampling a block extended with erasure coding was introduced by Al-Bassam *et al.* [8] and later adopted by LazyLedger [7], which evolved into Celestia [1]. Celestia employs a *centralized* approach in which validators—highly resourceful "super" nodes retrieve and store the complete blob data, while light clients sample from these nodes. Unlike PANDAS's collaborative model, where sampling and storage responsibilities are distributed, Celestia's design incurs overhead that grows linearly with participation and blob size. FullDAS [41], outlined in a research forum post, is another proposal aimed at scaling Danksharding to large blob sizes through gossip-based dissemination combined with a request/response sampling protocol. However, the sampling process is only sketched rather than specified with a concrete algorithm, and critical aspects such as how custody is assigned, what transport protocol is used, and the timing requirements for sampling remain underspecified, making it unsuitable as a baseline for experimental comparison.

Alternative sampling methods have also been explored. Honeybee [63] focuses on Sybil-resistant peer sampling via verifiable random walks, assuming random cells must be fetched from random peers. By contrast, PANDAS selects random cells but leverages their deterministic assignment to peers. Honeybee's interactive verification at each hop introduces potential latency, limiting its suitability for strict timing constraints. Sheng *et al.* [57] propose a different model in which a group of oracle nodes collectively store erasure-coded blobs and provide access to clients, assuming at least half of the oracles are honest.

11 Conclusion

We presented PANDAS, a practical approach to integrated data availability sampling (DAS) in the consensus workflow of Ethereum under the demanding Danksharding objectives. By favoring direct and lightweight communications between nodes, builder-led blob data dissemination, and adaptive fetching mechanisms, PANDAS allows large amounts of layer-2 data to propagate in the Ethereum network and be verified as available under the strict timing constraints imposed by Ethereum consensus.

This work opens interesting perspectives, among which is the design of *adaptive* policies. We presented and evaluated different fixed strategies for the builders and the nodes to follow. However, the design could support automatic adaptation mechanisms that select or update parameters based on, for example, observed networking and fault ratio conditions.

Acknowledgments: We thank our shepherd, Kaiwen Zhang, and the Middleware 2025 reviewers for their feedback. This work was supported by the Ethereum foundation (grant #FY22-0753), Belgian Wallonia CyberExcellence (grant #2110186), and CHIST-ERA/FNRS SCEAL projects.

References

[1] Celestia. the first modular blockchain network. https://celestia.org/.

[2] Ethereum full node vs. archive node. https://www.quicknode.com/guides/infrastructure/node-setup/ethereum-full-node-vs-archive-node.

[3] Ethereum node tracker. https://etherscan.io/nodetracker.

[4] Go Ethereum. Official Go implementation of the Ethereum protocol. https://geth.ethereum.org/.

[5] libp2p: A modular network stack. https://libp2p.io.

[6] EIP-7870: Hardware and bandwidth recommendations. https://eips.ethereum.org/EIPS/eip-7870, January 2025.

[7] Mustafa Al-Bassam. LazyLedger: A distributed data availability ledger with client-side smart contracts. *arXiv preprint arXiv:1905.09274*, 2019.

[8] Mustafa Al-Bassam, Alberto Sonnino, Vitalik Buterin, and Ismail Khoffi. Fraud and data availability proofs: Detecting invalid blocks in light clients. In *Intl. Conf. on Financial Cryptography and Data Security*, FC. Springer, 2021.

[9] Juan Benet. IPFS: content addressed, versioned, P2P file system. *arXiv preprint arXiv:1407.3561*, 2014.

[10] Vitalik Buterin, Dankrad Feist, Diederik Loerakker, George Kadianakis, Matt Garnett, Mofi Taiwo, and Ansgar Dietrichs. EIP-4844: Shard blob transactions. https://eips.ethereum.org/EIPS/eip-4844, 2024.

[11] Mikel Cortes-Goicoechea and Leonardo Bautista-Gomez. Discovering the Ethereum2 P2P network. In *2021 Third International Conference on Blockchain Computing and Applications*, BCCA. IEEE, 2021.

[12] Mikel Cortes-Goicoechea, Csaba Kiraly, Dmitriy Ryajov, Jose Luis Muñoz-Tapia, and Leonardo Bautista-Gomez. Scalability limitations of Kademlia DHTs when enabling Data Availability Sampling in Ethereum, 2024.

[13] Francesco D'Amato. From 4844 to Danksharding: a path to scaling Ethereum DA. https://ethresear.ch/t/from-4844-to-danksharding-a-path-to-scaling-ethereum-da, December 2023.

[14] Francesco D'Amato and Ansgar Dietrichs. SubnetDAS - an intermediate DAS approach. https://ethresear.ch/t/subnetdas-an-intermediate-das-approach/17169, 2023.

[15] Francesco D'Amato and Luca Zanolini. Recent latest message driven ghost: Balancing dynamic availability with asynchrony resilience. In *2024 IEEE 37th Computer Security Foundations Symposium (CSF)*, pages 127–142. IEEE, 2024.

[16] Francesco D'Amato, Luca Zanolini, and Roberto Saltini. DAS fork-choice. https://ethresear.ch/t/das-fork-choice/19578, May 2024.

[17] Ethereum. Randao: Ethereum random number generator. https://github.com/randao/randao, 2022.

[18] Ethereum. Zero-knowledge rollups. https://ethereum.org/en/developers/docs/scaling/zk-rollups/, 2024.

[19] Ethereum. Discv4 ENR periodic crawls. https://github.com/ethereum/discv4-dns-lists, 2025.

[20] Ethereum community. DAS query analysis notebook. https://colab.research.google.com/drive/1Di1-hBae8tZr1tZqcu1JqYycOFy8FdAy, 2024.

[21] Ethereum foundation. The Merge: Ethereum switch to proof-of-stake. https://ethereum.org/en/upgrades/merge/, 2023.

[22] Ethereum foundation. Ethereum roadmap. https://ethereum.org/en/roadmap/, 2024.

[23] Ethereum foundation. Ethereum roadmap: Danksharding. https://ethereum.org/en/roadmap/danksharding/, 2024.

[24] Ethereum foundation. Ethereum roadmap: Proposer-builder separation. https://ethereum.org/en/roadmap/pbs/, 2024.

[25] Ethereum foundation. Optimistic rollups. https://ethereum.org/en/developers/docs/scaling/optimistic-rollups/, 2024.

[26] Dankrad Feist. Proofs of custody. https://dankradfeist.de/ethereum/2021/09/30/proofs-of-custody.html, 2021.

[27] Dankrad Feist. New sharding design with tight beacon and shard block integration. https://notes.ethereum.org/@dankrad/new_sharding, 2022.

[28] Flashbots. MEV-Boost in a Nutshell. https://boost.flashbots.net, 2024.

[29] Ethereum Foundation. Node discovery protocol v5 - wire protocol. https://github.com/ethereum/devp2p/blob/master/discv5/discv5-wire.md#udp-communication.

[30] Ankit Gangwal, Haripriya Ravali Gangavalli, and Apoorva Thirupathi. A survey of Layer-two blockchain protocols. *Journal of Network and Computer Applications*, 209, 2023.

[31] Vincent Gramlich, Dennis Jelito, and Johannes Sedlmeir. Maximal extractable value: Current understanding, categorization, and open research questions. *Electronic Markets*, 34(1):49, 2024.

[32] Abdelatif Hafid, Abdelhakim Senhaji Hafid, and Mustapha Samih. Scaling blockchains: A comprehensive survey. *IEEE access*, 8:125244–125262, 2020.

[33] Lioba Heimbach, Lucianna Kiffer, Christof Ferreira Torres, and Roger Wattenhofer. Ethereum's proposer-builder separation: Promises and realities. In *Internet Measurement Conference*, IMC, pages 406–420. ACM.

[34] Lioba Heimbach, Yann Vonlanthen, Juan Villacis, Lucianna Kiffer, and Roger Wattenhofer. Deanonymizing Ethereum validators: The P2P network has a privacy issue. In *USENIX Security Symposium*, 2025.

[35] Sebastian Henningsen, Daniel Teunis, Martin Florian, and Björn Scheuermann. Eclipsing Ethereum peers with false friends. In *2019 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pages 300–309. IEEE, 2019.

[36] Cornelius Ihle, Dennis Trautwein, Moritz Schubotz, Norman Meuschke, and Bela Gipp. Incentive mechanisms in peer-to-peer networks—a systematic literature review. *ACM Computing Surveys*, 55(14s):1–69, 2023.

[37] Maxim Jourenko, Kanta Kurazumi, Mario Larangeira, and Keisuke Tanaka. SoK: A taxonomy for Layer-2 scalability related protocols for cryptocurrencies. Cryptology ePrint Archive, Paper 2019/352, 2019.

[38] George Kadianakis, Mary Maller, Andrija Novakovic, and Suphanat Chunhapanya. Proof of Validator: A simple anonymous credential scheme for Ethereum's DHT. https://ethresear.ch/t/proof-of-validator-a-simple-anonymous-credential-scheme-for-ethereums-dht/16454, August 2023.

[39] Harry Kalodner, Steven Goldfeder, Xiaoqi Chen, S Matthew Weinberg, and Edward W Felten. Arbitrum: Scalable, private smart contracts. In *27th USENIX Security Symposium*, 2018.

[40] Aniket Kate, Gregory M Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In *Intl. conf. on the theory and app. of cryptology and info. sec.*, ASIACRYPT, 2010.

[41] Csaba Kiraly, Leonardo Bautista-Gomez, and Dmitriy Ryajov. FullDAS - towards massive scalability with 32MB blocks and beyond https://ethresear.ch/t/fulldas-towards-massive-scalability-with-32mb-blocks-and-beyond/19529/1, May 2024.

[42] Michał Król, Onur Ascigil, Sergi Rene, Alberto Sonnino, Matthieu Pigaglio, Ramin Sadre, Felix Lange, and Etienne Riviere. Disc-NG: Robust service discovery in the Ethereum Global Network. In *9th European Symposium on Security and Privacy*, EuroS&P, pages 193–215. IEEE, 2024.

[43] Probe Lab. Final report: NAT hole punching measurement campaign. https://github.com/plprobelab/network-measurements/blob/master/results/rfm15-nat-hole-punching.md, 2023.

[44] Labrys.io. MEV Watch. https://www.mevwatch.info, 2024.

[45] Yuval Marcus, Ethan Heilman, and Sharon Goldberg. Low-resource eclipse attacks on Ethereum's peer-to-peer network. *IACR Cryptology ePrint Archive*, 2018(236).

[46] Matter Labs. zkSync. https://zksync.io, 2024.

[47] Petar Maymounkov and David Mazieres. Kademlia: A peer-to-peer information system based on the XOR metric. In *International Workshop on Peer-to-Peer Systems*, IPTPS. Springer, 2002.

[48] Alberto Montresor and Márk Jelasity. PeerSim: A scalable P2P simulator. In *Proc. of the 9th Int. Conference on Peer-to-Peer*, P2P, 2009.

[49] Optimism. https://www.optimism.io, 2024.

[50] Ulysse Pavloff, Yackolley Amoussou-Guenou, and Sara Tucci-Piergiovanni. Byzantine attacks exploiting penalties in Ethereum PoS. In *2024 54th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 53–65. IEEE, 2024.

[51] Matthieu Pigaglio, Onur Ascigil, Michał Król, Sergi Rene, Felix Lange, Kaleem Peeroo, Ramin Sadre, Vladimir Stankovic, and Etienne Rivière. PANDAS prototype and simulator. https://github.com/CloudLargeScale-UCLouvain/PANDAS, 2025.

[52] Polygon. https://polygon.technology, 2024.

[53] David Powell. Failure mode assumptions and assumption coverage. In *Predictably dependable computing systems*, pages 123–140. Springer, 1995.

[54] Gabriel Antonio F Rebello, Gustavo F Camilo, Lucas Airam C de Souza, Maria Potop-Butucaru, Marcelo Dias de Amorim, Miguel Elias M Campista, and Luís Henrique MK Costa. A survey on blockchain scalability: From hardware to layer-two protocols. *IEEE Communications Surveys & Tutorials*, 2024.

[55] Codex.storage Research. Crawling the Ethereum DISCV5 network, fast. https://ethresear.ch/t/crawling-the-ethereum-discv5-network-fast/20962, 2024.

[56] Danny Ryan. PeerDAS – a simpler DAS approach using battle-tested P2P components. https://ethresear.ch/t/peerdas-a-simpler-das-approach-using-battle-tested-p2p-components/16541, 2023.

[57] Peiyao Sheng, Bowen Xue, Sreeram Kannan, and Pramod Viswanath. ACeD: Scalable data availability oracle. In *Financial Cryptography and Data Security*, FC. Springer, 2021.

[58] Srivatsan Sridhar, Onur Ascigil, Navin Keizer, François Genon, Sébastien Pierre, Yiannis Psaras, Etienne Rivière, and Michał Król. Content censorship in the interplanetary file system. In *Network and Distributed System Security Symposium*, NDSS, 2024.

[59] Dennis Trautwein. Nebula: A network agnostic DHT crawler and monitor. https://github.com/dennis-tra/nebula, 2025.

[60] Dimitris Vyzovitis, Yusef Napora, Dirk McCormick, David Dias, and Yiannis Psaras. Gossipsub: Attack-resilient message propagation in the Filecoin and ETH2.0 networks. *arXiv preprint arXiv:2007.02754*, 2020.

[61] Stephen B Wicker and Vijay K Bhargava. *Reed-Solomon codes and their applications*. John Wiley & Sons, 1999.

[62] Karl Wüst and Arthur Gervais. Ethereum eclipse attacks. Technical report, ETH Zurich, 2016.

[63] Yunqi Zhang and Shaileshh Bojja Venkatakrishnan. Honeybee: Decentralized peer sampling with verifiable random walks for blockchain data sharding. *arXiv e-prints*, pages arXiv–2402, 2024.